



Decentralized Application Code Review And Security Analysis Report

Customer: Sundial

Date: 17/04/2026



We express our gratitude to the Sundial team for the collaborative engagement that enabled the execution of this dApp Security Assessment.

Sundial Protocol is building Bitcoin-native yield infrastructure, enabling users to lock BTC into time-based escrow contracts and earn yield through a decentralized staking mechanism. The protocol leverages Bitcoin Script primitives (CLTV, CSV) to create trustless timelock and escrow outputs, allowing for non-custodial yield distribution without requiring wrapped tokens or cross-chain bridges.

Document

Name	dApp Code Review and Security Analysis Report for Sundial
Audited By	Adedolapo Olayinka-Adeyemi
Approved By	Ece Orsel
Website	sundialprotocol.com
Changelog	30/01/2025 - Preliminary Report
Platform	Bitcoin
Language	Typescript
Tags	White-box
Methodology	https://docs.hacken.io/methodologies/dapp-audit-methodology

Review Scope

Repository	https://github.com/sundial-protocol/btc-locker
Commit	dd1332516d9721b67fefdc219048527de066cbdc



Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

11	11	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	3
Low	4

Vulnerability	Severity	Status
F-2026-16093 - Infinite Recursion in BTCLocker Facade Methods	Medium	Fixed
F-2026-16094 - Incorrect PSBT Finalization Logic for Script-Based Spending	Medium	Fixed
F-2026-16096 - Spending PSBT Builder Omits Required Previous Output Data	Medium	Fixed
F-2026-16095 - Inconsistent PSBT Encoding Across Public API	Low	Fixed
F-2026-16097 - Relative Timelock (CSV) Exposed but Not Fully Implemented	Low	Fixed
F-2026-16098 - Distribution Builder Uses Incorrect Input Script Fallback	Low	Fixed
F-2026-16099 - Claim and Withdraw Flows Assume P2SH While Address Uses SegWit-Like Form	Low	Fixed
F-2026-16083 - Incorrect Payment Template for Script Address Generation	Info	Fixed
F-2026-16115 - UTXO Selection Without Ordinal-Safe Controls	Info	Fixed
F-2026-16116 - Keypair Object Serialization Exposes Internal ECPair Properties	Info	Fixed
F-2026-16117 - Transaction Amount Lacks Upper Bound Validation	Info	Fixed

Documentation quality

- Comprehensive top-level `README.md` with protocol overview, installation instructions, and usage examples
- Detailed `CLI.md` documenting all command-line operations with flags and examples

Code quality

- Strong TypeScript usage with strict typings, dedicated `types.ts`, and interface definitions for all transaction parameters
- Clean modular architecture with single-responsibility separation (`locker/scripts/`, `locker/transactions/`, `utils/`)

Table of Contents

System Overview	6
Findings	8
Vulnerability Details	8
F-2026-16096 - Spending PSBT Builder Omits Required Previous Output Data - Medium	8
F-2026-16094 - Incorrect PSBT Finalization Logic For Script-Based Spending - Medium	10
F-2026-16093 - Infinite Recursion In BTCLocker Facade Methods - Medium	12
F-2026-16098 - Distribution Builder Uses Incorrect Input Script Fallback - Low	14
F-2026-16095 - Inconsistent PSBT Encoding Across Public API - Low	16
F-2026-16097 - Relative Timelock (CSV) Exposed But Not Fully Implemented - Low	18
F-2026-16099 - Claim And Withdraw Flows Assume P2SH While Address Uses SegWit-Like Form - Low	20
F-2026-16083 - Incorrect Payment Template For Script Address Generation - Info	22
F-2026-16115 - UTXO Selection Without Ordinal-Safe Controls - Info	24
F-2026-16117 - Transaction Amount Lacks Upper Bound Validation - Info	26
F-2026-16116 - Keypair Object Serialization Exposes Internal ECPair Properties - Info	27
Disclaimers	29
Hacken Disclaimer	29
Technical Disclaimer	29
Appendix 1. Severity Definitions	30
Appendix 2. Scope	31
Appendix 3. Additional Valuables	32
Frameworks And Methodologies	32

System Overview

The BTC Locker core library lives in its own top-level `/packages/core` folder and is responsible for Bitcoin script generation, PSBT construction, transaction signing, and on-chain interaction with Bitcoin APIs. Inside there is:

A `README.md` and supporting documentation that explain overall setup, usage patterns, and integration guidance for the Sundial staking workflow.

`src/` with several subdirectories and standalone modules:

- `locker/` — the primary module containing Bitcoin locking primitives:
 - `core.ts` which provides the `BTCLockerCore` base class handling ECC initialization, transaction signing, and PSBT finalization logic.
 - `index.ts` exposing the unified `BTCLocker` facade class that combines key generation, script management, and transaction building into a single interface.
 - `keypair.ts` for Bitcoin key pair generation (random and from existing private keys) using the secp256k1 curve.
 - `script-manager.ts` and `transaction-manager.ts` as thin facades delegating to specialized builders.
 - `scripts/` subdirectory containing:
 - `timelock.ts` for absolute (CLTV) and relative (CSV) timelock script creation.
 - `escrow.ts` for dual-path escrow scripts with before/after deadline spending conditions.
 - `transactions/` subdirectory containing:
 - `deposit/` with `deposit.ts`, `deposit-with-script.ts`, and `calculate.ts` for Dawn Protocol staking transactions that split funds between escrow and timelock outputs.
 - `withdraw.ts` for combining escrow and timelock inputs into a single withdrawal transaction.
 - `claim.ts` for spending from escrow scripts using either the before-deadline or after-deadline path.
 - `distribute.ts` for yield distribution transactions back to timelock addresses.
 - `generic.ts` providing low-level spending and funding transaction primitives.
- `utils/` — shared utility modules:
 - `scripts.ts` with helpers for script validation, address generation, script parsing, and locktime extraction.
 - `keys.ts` for public/private key validation and ECPair creation.
 - `fees.ts` encapsulating fee estimation logic with priority levels (HIGH/MEDIUM/LOW) and dust threshold calculations.
 - `transactions.ts` for UTXO selection, witness input construction, and metadata output appending.
 - `metadata.ts` implementing the Sundial OP_RETURN metadata protocol (60-byte schema with magic bytes, version, transaction type, deposit ID, provider pubkey, flags, and CRC-32 checksum).
 - `validation.ts` with common parameter validation (locktime, amounts, protocol fees).
 - `time.ts` for timestamp/date conversions and duration calculations.
 - `network.ts` defining Bitcoin network configurations (mainnet, testnet, regtest).

- `bitcoin-api.ts` — integration layer for external Bitcoin APIs (Mempool.space, Blockstream, BlockCypher) providing UTXO fetching, transaction broadcasting, fee estimation, and block height queries.
- `errors.ts` defining custom error classes (`BTCLockerError` , `ValidationError` , `TimelockError`) for structured error handling.
- `types.ts` and `index.ts` for TypeScript type definitions and public API exports.

Supporting files include `package.json` , TypeScript configurations (`tsconfig.json` , `tsconfig.build.json`), Vitest test suite under `tests/` , ESLint/Prettier configs, and webpack bundling configuration for browser distribution.

Findings

Vulnerability Details

[F-2026-16096](#) - Spending PSBT Builder Omits Required Previous Output Data - Medium

Description:

The generic `createSpendingTransaction` function adds PSBT inputs containing only hash, index, and sequence. It does not include `witnessUtxo` or `nonWitnessUtxo`. BIP-174 requires each input to contain previous-output information so that signers can verify what they are signing.

This is inconsistent with every other transaction builder in the library. For comparison, `claim.ts:91-117` correctly fetches and attaches the full previous transaction hex as `nonWitnessUtxo`.

Status:

Fixed

Classification

CVSS 4.0:

5.4 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:L/SC:N/SI:N/SA:N)

Severity:

Medium

Recommendations

Remediation:

- Require/fetch prevout data and populate `witnessUtxo` or `nonWitnessUtxo` for every input.

Evidences

Proof of Concept

Location:

`packages/core/src/locker/transactions/generic.ts:121-127`

Reproduce:

```
inputs.forEach((utxo) => {
  const txHash = Buffer.from(utxo.txid, "hex").reverse();
  psbt.addInput({
    hash: txHash,
    index: utxo.vout,
    sequence: locktime ? 0xffffffff : 0xffffffff,
    // NO nonWitnessUtxo
    // NO witnessUtxo
  });
});
```



```
// NO redeemScript
});
});
```

```
Compare with claim.ts:91–117 (correct):
inputData = {
  hash: utxoTxId,
  index: utxoIndex,
  sequence,
  redeemScript,
  nonWitnessUtxo: Buffer.from(txHex, "hex"), // correctly populated
};
```

- Call `createSpendingTransaction` with valid inputs, outputs, and redeemScript.
- Inspect the returned PSBT — each input has only outputpoint data.
- Pass the PSBT to any compliant signer (hardware wallet, bitcoinjs-lib, etc.).
- Signer refuses to sign or throws because prevout verification data is missing.
- Transaction is permanently unsigned and unbroadcastable.

[F-2026-16094](#) - Incorrect PSBT Finalization Logic for Script-Based Spending - Medium

Description: The `finalizeTransaction` method in `BTCLockerCore` applies a custom PSBT finalization callback for all script inputs. The finalization always populates `finalScriptSig` and explicitly sets `finalScriptWitness`: undefined.

For simple timelock scripts, finalization produces: `scriptSig = [<signature>, <redeemScript>]`. For escrow scripts with conditional logic, finalization produces: `scriptSig = [<signature>, OP_TRUE/OP_FALSE, <redeemScript>]`.

These patterns are only valid for legacy `P2SH` outputs. The library also attempts to create SegWit-like addresses and uses `witnessUtxo` in some code paths. For any witness-based spend (P2WSH, P2SH-P2WSH), the correct finalization must populate the witness stack, not `scriptSig`.

Status: Fixed

Classification

CVSS 4.0: 5.2 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:H/SC:N/SI:N/SA:N)

Severity: Medium

Recommendations

Remediation:

- Finalization logic must be conditioned on the actual payment type:

```
- P2SH → `finalScriptSig = [<sig>, <redeemScript>]`, no witness
- P2WSH → `finalScriptWitness = [<sig>, <witnessScript>]`, no scriptSig
- P2SH-P2WSH → `finalScriptSig = [<p2wsh-program>]` + `finalScriptWitness = [<sig>, <witnessScript>]`
```

- Use `bitcoin.payments.*` to derive finalization helpers from the same canonical payment object used to generate the address.

Evidences

Proof of Concept

Location:

packages/core/src/locker/core.ts:231–285,
packages/core/src/locker/transactions/claim.ts:88–119,
packages/core/src/locker/transactions/withdraw.ts:153–193

Reproduce:

Source: packages/core/src/locker/core.ts:246–263

```
psbt.finalizeInput(i, (inputIndex: number, inputData: any) => {  
  const signature = inputData.partialSig?.[0]?.signature;  
  const scriptSig = bitcoin.script.compile([  
    signature,  
    spendAfterDeadline ? bitcoin.opcodes.OP_TRUE : bitcoin.opcodes.OP_F  
    ALSE,  
    redeemScript,  
  ]);  
  return {  
    finalScriptSig: scriptSig, // P2SH path only  
    finalScriptWitness: undefined, // explicitly no witness – BROKEN fo  
    r P2WSH  
  };  
});
```

- Create an escrow script and deposit funds to its address.
- Call `createClaimTransaction()` to build a spending PSBT.
- Call `signTransaction()` signing succeeds in memory.
- `finalizeTransaction` is called internally; it builds a legacy scriptSig.
- `finalScriptWitness` is set to undefined.
- `extractTransaction()` produces a transaction without witness data.
- If the address was SegWit (as generated by F01), network nodes will reject the transaction.

[F-2026-16093](#) - Infinite Recursion in BTCLocker Facade Methods - Medium

Description: The BTCLocker class serves as the public facade for the entire library. It contains two transaction-building methods which are `createSpendingTransaction` and `createFundingTransaction` that call themselves recursively instead of delegating to `this.transactionManager`.

Every other method in the same class correctly delegates (e.g., `createClaimTransaction` → `this.transactionManager.createClaimTransaction(params)`). The two affected methods are the only ones with this defect, making it an implementation oversight. Facade methods recursively call themselves instead of delegating to `transactionManager`.

Status: Fixed

Classification

CVSS 4.0: 5.1 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:N/VA:H/SC:N/SI:N/SA:N)

Severity: Medium

Recommendations

Remediation:

- Replace both self-calls with proper delegation:

```
// Fix createSpendingTransaction
async createSpendingTransaction(params: SpendingTransactionParams):
Promise<string> {
  return this.transactionManager.createSpendingTransaction(params); //
  / was: this.createSpendingTransaction(params)
}
```

```
// Fix createFundingTransaction
async createFundingTransaction(params: FundingTransactionParams): P
romise<string> {
  return this.transactionManager.createFundingTransaction(params); //
  was: this.createFundingTransaction(params)
}
```

- Add unit tests that invoke both methods directly on a BTCLocker instance to prevent regression.

Evidences

Proof of Concept

Location: packages/core/src/locker/index.ts:166–170

Reproduce:

```
async createSpendingTransaction(  
  params: SpendingTransactionParams,  
): Promise<string> {  
  return this.createSpendingTransaction(params); // SELF-CALL – infinite recursion  
}
```

Source: packages/core/src/locker/index.ts:188–192

```
async createFundingTransaction(  
  params: FundingTransactionParams,  
): Promise<string> {  
  return this.createFundingTransaction(params); // SELF-CALL – infinite recursion  
}
```

```
Compare with a correctly delegating method at line 265:  
async createClaimTransaction(params: ClaimParams): Promise<string>  
{  
  return this.transactionManager.createClaimTransaction(params); // CORRECT  
}
```

- Instantiate const locker = new BTCLocker('testnet').
- Call `await locker.init()`.
- Call await locker.createSpendingTransaction({ inputs: [...], outputs: [...], redeemScript: '...' }).
- The method immediately calls `this.createSpendingTransaction(params)` itself.
- This repeats until the JavaScript call stack is exhausted.
- Runtime throws: RangeError: Maximum call stack size exceeded.

[F-2026-16098](#) - Distribution Builder Uses Incorrect Input Script

Fallback - Low

Description:

`createDistributionTransaction` builds a PSBT that spends from a provider's source address and outputs to a timelock address. When explicit inputs are provided but `sourceAddress` is omitted, the fallback script passed to `addWitnessInputs` is derived from `sourceAddress || timelockAddress`.

Since `sourceAddress` is undefined in this path, the fallback resolves to `timelockAddress` the destination of the distribution, not the source. PSBT inputs must carry the scriptPubKey of their actual prevout. Using the destination script as the input's `witnessUtxo.script` produces a malformed PSBT.

Status:

Fixed

Classification

CVSS 4.0: 4.2 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:L/SC:N/SI:N/SA:N)

Severity:

Low

Recommendations

Remediation:

When explicit inputs are provided, require either:

- Each input to carry its own scriptPubKey field (already supported in the UTXO type), or
- `sourceAddress` to be provided explicitly

Do not fall back to the destination address to infer input script context.

Evidences

Proof of Concept

Location:

`packages/core/src/locker/transactions/distribute.ts:82-89`

Reproduce:

Source: `packages/core/src/locker/transactions/distribute.ts:82-89`

```
TransactionUtils.addWitnessInputs(  
  psbt,  
  inputs,  
  bitcoin.address.toOutputScript(  
    sourceAddress || timelockAddress, // timelockAddress is the DESTINA
```

```
TION
ctx.network,
),
);
```

- Call `createDistributionTransaction` with explicit inputs from a different address.
- Omit `sourceAddress`.
- Fallback resolves to `timelockAddress` (destination).
- Each PSBT input's `witnessUtxo.script` is set to the timelock address's `scriptPubKey`.
- The actual prevouts are from a different address with a different script.
- Signers verify that `witnessUtxo.script` matches the actual prevout, they will reject the mismatch.
- PSBT is malformed and the transaction cannot be signed.

[F-2026-16095](#) - Inconsistent PSBT Encoding Across Public API - Low

Description:

The library's transaction builders return unsigned PSBTs in two different encodings: `createSpendingTransaction` returns `psbt.toHex()` while `createFundingTransaction` and all other builders return `psbt.toBase64()`. The single signing function `signTransaction` always parses the input using `bitcoin.Psbt.fromBase64()`.

A caller who follows the natural API pattern to build spending transaction, then sign it, receives a hex-encoded PSBT from the first call and attempts to pass it directly into the second call, which expects base64.

Spending and funding builders return different PSBT encodings, but signing expects base64 only.

Status:

Fixed

Classification

CVSS 4.0:

4.0 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:H/SC:N/SI:N/SA:N)

Severity:

Low

Recommendations

Remediation:

- Standardize unsigned PSBT outputs to base64 across all builders.
- Change `createSpendingTransaction` to return `psbt.toBase64()`.
- Reserve raw hex only for fully signed and extracted final transactions.

Evidences

Proof of Concept

Location:

packages/core/src/locker/transactions/generic.ts:136,
packages/core/src/locker/transactions/generic.ts:161,
packages/core/src/locker/core.ts:172

Reproduce:

```
Source: packages/core/src/locker/transactions/generic.ts:136  
return psbt.toHex(); // createSpendingTransaction returns HEX
```

```
Source: packages/core/src/locker/transactions/generic.ts:161  
return psbt.toBase64(); // createFundingTransaction returns BASE64
```

```
Source: packages/core/src/locker/core.ts:172  
const psbt = bitcoin.Psbt.fromBase64(unsignedPsbt, { network: this.
```



```
network });  
// signTransaction always deserializes as BASE64
```

- Call `createSpendingTransaction()`.
- Pass returned string to `signTransaction()`.
- `fromBase64` parse fails for hex payload.

[F-2026-16097](#) - Relative Timelock (CSV) Exposed but Not Fully Implemented - Low

Description:

`createRelativeTimelockScript` creates a script containing `OP_CHECKSEQUENCEVERIFY` (CSV) and exposes it as a public API method. However, CSV spending has additional requirements that are not implemented:

1. **Transaction version ≥ 2** - is required by BIP-68 and the library does not enforce this.
2. **Correct nSequence encoding** — the sequence must encode the relative locktime in BIP-68 format (block-based vs time-based, disable flag bit). The library uses a hardcoded `0xfffffffffe` or `0xfffffffff`.
3. **extractLocktimeFromScript** — only searches for `OP_CHECKLOCKTIMEVERIFY`, not `OP_CHECKSEQUENCEVERIFY`. CSV scripts are not detected and their sequence numbers are not validated at spend time.

Status:

Fixed

Classification

CVSS 4.0:

4.0 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:L/SC:N/SI:N/SA:N)

Severity:

Low

Recommendations

Remediation:

- Fully implement CSV support, including BIP-68 sequence encoding, transaction version 2 enforcement, and a CSV-aware `extractSequenceFromScript` function.
- Remove `createRelativeTimelockScript` from the public API and mark it as unsupported until complete.

Evidences

Proof of Concept

Location:

packages/core/src/locker/scripts/timelock.ts:50–79,
packages/core/src/utils/scripts.ts:136–153,
packages/core/src/locker/transactions/generic.ts:100–127

Reproduce:

```
Source: packages/core/src/utls/scripts.ts:140-142
const cltvIndex = ops.findIndex(
  (op) => op === bitcoin.opcodes.OP_CHECKLOCKTIMEVERIFY, // only checks CLTV, not CSV
);

Source: packages/core/src/locker/transactions/generic.ts:120-123
// Locktime extraction returns null for CSV scripts
const locktime = ScriptUtils.extractLocktimeFromScript(redeemScript);
// null for CSV
// ...
sequence: locktime ? 0xffffffff : 0xffffffff, // CSV sequence semantics not applied
```

- Create a CSV script: `locker.createRelativeTimelockScript(144, publicKey)` (144 blocks).
- Build a spending PSBT using `createSpendingTransaction`.
- The locktime extractor returns null (no CLTV found in script).
- Sequence is set to `0xffffffff` (final/non-relative-locktime).
- The transaction violates BIP-68 nSequence requirements for CSV scripts.
- Bitcoin nodes reject the transaction.

[F-2026-16099](#) - Claim and Withdraw Flows Assume P2SH While Address Uses SegWit-Like Form - Low

Description:

The address generator produces a P2WPKH-like bech32 address using a script hash. The claim and withdrawal builders construct PSBTs using `nonWitnessUtxo` and `redeemScript`. The correct fields for legacy P2SH inputs. These two models are mutually incompatible.

A P2WPKH output (as produced by the address generator) must be spent with `witnessUtxo` (not `nonWitnessUtxo`), and the witness must contain [`<signature>`, `<33-byte-compressed-pubkey>`] not a `redeemScript`. There is no single coherent payment descriptor used across the create and spend paths.

Status:

Fixed

Classification

CVSS 4.0:

4.0 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:L/SC:N/SI:N/SA:N)

Severity:

Low

Recommendations

Remediation:

- Implement one consistent payment template (P2SH or P2WSH) throughout, using the same payment object to derive both the address and the PSBT input fields.
- This finding is resolved as part of fixing F2026-16083

Evidences

Proof of Concept

Location:

packages/core/src/utls/scripts.ts:101–105,
packages/core/src/locker/transactions/claim.ts:88–119,
packages/core/src/locker/transactions/withdraw.ts:153–193

Reproduce:

```
Address generation (scripts.ts:101-105):  
p2wpkh({ hash: hash160(redeemScript) }) → tb1q... SegWit-like address
```

```
Claim PSBT input (claim.ts:91-96):  
{  
  hash: utxoTxId,
```

```
index: utxoIndex,  
nonWitnessUtxo: Buffer.from(txHex, "hex"), // P2SH pattern  
redeemScript, // P2SH pattern  
sequence,  
}
```

- P2WPKH addresses require `witnessUtxo`, not `nonWitnessUtxo`. The library mixes both models without a consistent payment type.

[F-2026-16083](#) - Incorrect Payment Template for Script Address

Generation - Info

Description:

The function `createScriptAddress()` is responsible for generating the Bitcoin address that corresponds to each redeem script (timelock, relative timelock, escrow). The implementation computes `hash160(redeemScript)` which is the correct first step for a script-hash address but then passes this value into `bitcoin.payments.p2wpkh({ hash: scriptHash })`.

P2WPKH (Pay-to-Witness-Public-Key-Hash) is a SegWit v0 payment type intended for a 20-byte public key hash, not a 20-byte script hash. For script-locked outputs the correct payment templates are:

- `bitcoin.payments.p2sh({ redeem: })` — for legacy P2SH
- `bitcoin.payments.p2wsh({ redeem: })` — for native SegWit P2WSH

This creates a fundamental mismatch: addresses emitted by the library represent a P2WPKH output (SegWit v0, bc1q... / tb1q bech32), but to spend a P2WPKH output you must provide the matching compressed public key not a redeem script. The intended redeem script can never unlock those addresses.

Script-locked outputs are being encoded through `p2wpkh` using a redeem script hash, which is a payment-template mismatch.

Status:

Fixed

Classification

CVSS 4.0:

8.1 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:H/VA:H/SC:N/SI:N/SA:N)

Severity:

Info

Recommendations

Remediation:

- Replace the `p2wpkh` call with the correct payment template. Choose one model and implement it consistently:

```
// Option A: Legacy P2SH
const payment = bitcoin.payments.p2sh({
  redeem: { output: redeemScript, network },
  network,
});
```

```
// Option B: Native SegWit P2WSH
const payment = bitcoin.payments.p2wsh({
```

```
redeem: { output: redeemScript, network },
network,
});
```

- Derive addresses, PSBT input fields, and finalization logic from the same `payment` object throughout the library.

Evidences

Proof of Concept

Location: packages/core/src/utls/scripts.ts:101–105

Reproduce:

```
static createScriptAddress(redeemScript: Buffer, network: bitcoin.N
etwork): string {
  const scriptHash = bitcoin.crypto.hash160(redeemScript); // line 10
  1
  const payment = bitcoin.payments.p2wpkh({ // line 102 – WRONG TYPE
    hash: scriptHash,
    network: network,
  });
  return payment.address; // produces bclq.../tb1q... bech32 P2WPKH a
ddress
}
```

- Call `locker.createTimelockScript(locktime, publicKey)`.
- Internally `createScriptAddress` is called on the compiled CLTV redeem script.
- `hash160(redeemScript)` produces a correct 20-byte hash of the script.
- That hash is passed to `p2wpkh`, which generates output script: ``OP_0 <20-byte-hash>``.
- The resulting `tb1q...` address is a valid bech32 address, but it locks funds to a P2WPKH output.
- To spend P2WPKH, a node expects witness: [`<signature>`, `<33-byte-compressed-pubkey>`].
- The library instead attempts to spend using a `redeemScript` in `scriptSig` (see F02).
- The output is unspendable via the library's own spending path.

[F-2026-16115](#) - UTXO Selection Without Ordinal-Safe Controls - Info

Description:

`TransactionUtils.selectUtxos` selects UTXOs by sorting descending by value and consuming until the target amount is met. There is no mechanism to exclude specific UTXOs, no ordinal/inscription awareness, no wallet segregation, and no manual coin control.

In a Bitcoin wallet that holds inscriptions or ordinal-sensitive UTXOs alongside protocol funds, the greedy algorithm may select and irreversibly spend an inscription UTXO as fee or change burn.

Status:

Fixed

Classification

CVSS 4.0: 1.0 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:L/VA:N/SC:N/SI:N/SA:N)

Severity:

Info

Recommendations

Remediation:

- Add an exclusion list parameter to `selectUtxos` and extend the UTXO interface with an optional `doNotSpend`: boolean flag.
- Provide wallet-segregation guidance for protocol vs. general wallet funds.

Evidences

Proof of Concept

Location:

`packages/core/src/utls/transactions.ts:31-53`

Reproduce:

Source: `packages/core/src/utls/transactions.ts:31-53`

```
static selectUtxos(availableUtxos: UTXO[], targetAmount: number): UTXO[] {
  const sortedUtxos = [...availableUtxos].sort((a, b) => b.value - a.value);
  // Picks highest value UTXOs first – no exclusion mechanism
  for (const utxo of sortedUtxos) {
    selectedUtxos.push(utxo);
    totalValue += utxo.value;
    if (totalValue >= targetAmount) break;
  }
}
```


The UTXO interface (types.ts:56-62) has no reserved, doNotSpend, or inscribed fields.

[F-2026-16117](#) - Transaction Amount Lacks Upper Bound Validation - Info

Description: ValidationUtils.validateAmount checks only that an amount is a positive integer. It does not enforce an upper bound. There is no check against `Number.MAX_SAFE_INTEGER` or the Bitcoin-specific maximum of 21,000,000 BTC (2,100,000,000,000,000 satoshis). JavaScript's Number type loses integer precision above $2^{53} - 1$ (approximately 9 quadrillion). Values above this threshold would silently produce incorrect arithmetic.

Status: Fixed

Classification

CVSS 4.0: 1.0 (/AV:N/AC:L/AT:N/PR:N/UI:N/VC:N/VI:L/VA:N/SC:N/SI:N/SA:N)

Severity: Info

Recommendations

Remediation:

- Add an upper bound check: `amount <= 2_100_000_000_000_000` (21M BTC in satoshis) and `amount <= Number.MAX_SAFE_INTEGER`.

Evidences

Proof of Concept

Location: packages/core/src/utils/validation.ts:37–52

Reproduce:

```
static validateAmount(amount, paramName = "amount", allowZero = false) {
  const amountNumber = Number(amount);
  assertAll([
    [amount !== null, `${paramName} cannot be undefined or null`],
    [Number.isInteger(amountNumber), `${paramName} must be an integer`],
    [amountNumber > 0, `${paramName} must be positive`],
    // No upper bound check
  ]);
  return amountNumber;
}

// Passes without error:
validateAmount(Number.MAX_SAFE_INTEGER + 1) // precision loss, no error
validateAmount(9999999999999999999) // well above BTC max supply
```

[F-2026-16116](#) - Keypair Object Serialization Exposes Internal ECPair Properties - Info

Description:

`generateKeyPairFromPrivateKey` returns an `ExtendedKeyPair` object that includes a `keyPair` property containing the raw `ECPair` interface object from the `ecpair` library. The `/api/keypair/from-private-key` route serializes this entire object with `res.json(keyPair)`.

When `JSON.stringify` serializes an `ECPair` object, it may expose additional internal properties depending on the `ECPair` implementation's enumerable properties. At minimum, it exposes the `keyPair` field as an object, which may recursively expose curve parameters or internal state.

This is a narrower issue, it is about the inclusion of an internal implementation object in the API response contract, not just the private key itself.

Status:

Fixed

Classification

Severity:

Info

Recommendations

Remediation:

Explicitly project only the required fields before returning from the API route:

```
const keyPair = await locker.generateKeyPairFromPrivateKey(privateKey);
res.json({
  publicKey: keyPair.publicKey,
  address: keyPair.address,
  // Do not include privateKey or keyPair
});
```

Evidences

Proof of Concept

Location:

`packages/core/src/locker/keypair.ts:97–103`, `packages/demo/demo/api-routes.js:169–170`

Reproduce:

```
Source: packages/core/src/locker/keypair.ts:97–103
return {
```

```
privateKey: keyPair.privateKey.toString("hex"),
publicKey: keyPair.publicKey.toString("hex"),
address,
keyPair, // internal ECPair object – not a simple serializable value
};
```

Source: packages/demo/demo/api-routes.js:169–170

```
const keyPair = await locker.generateKeyPairFromPrivateKey(privateKey);
res.json(keyPair); // serializes including the internal keyPair field
```

Disclaimers

Hacken Disclaimer

The application given for audit has been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in the application's source code, its deployment, and functionality (performing the intended functions) being the focus of our analysis.

The report contains no statements or warranties regarding the identification of all vulnerabilities or the absolute security of the code. The report covers only the code that was submitted and reviewed, and therefore may not remain relevant after any modifications have been made. This report should not be considered a definitive or exhaustive assessment of the utility, safety, or bug-free status of the application, nor should it be taken as a guarantee of the absence of other potential issues.

While we have exerted our best efforts in conducting the analysis and producing this report, it is crucial to understand that this report should not be the sole source of reliance for ensuring the security of the application. We strongly recommend undertaking multiple independent audits and establishing a public bug bounty program to enhance the security posture of the application.

English is the original language of this report. The Consultant is not liable for any errors or omissions in any translations of this report.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Applications, whether decentralized apps (DApps) or other types of applications, are deployed and run within specific environments that may include various platforms, programming languages, and other related software components. These environments and components can have inherent vulnerabilities that might lead to security breaches. Consequently, the Consultant cannot guarantee the absolute security of the audited application.

Appendix 1. Severity Definitions

Findings are categorized based on their potential impact and assigned a severity level using the [Common Vulnerability Scoring System \(CVSS\) version 4.0](#):

Severity	Description
Critical	These issues present a major security vulnerability that poses a severe risk to the system. They require immediate attention and must be resolved to prevent a potential security breach or other significant harm.
High	These issues present a significant risk to the system, but may not require immediate attention. They should be addressed in a timely manner to reduce the risk of the potential security breach.
Medium	These issues present a moderate risk to the system and cannot have a great impact on its function. They should be addressed in a reasonable time frame, but may not require immediate attention.
Low	These issues present no risk to the system and typically relate to the code quality problems or general recommendations. They do not require immediate attention and should be viewed as a minor recommendation.

Appendix 2. Scope

The scope of the project includes the following:

Scope Details	
Repository	https://github.com/sundial-protocol/btc-locker
Commit	dd1332516d9721b67fefdc219048527de066cbdc

Appendix 3. Additional Valuables

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), the [Penetration Testing Execution Standard \(PTES\)](#), and the [OWASP Testing Guide](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.